Assignment No.4

Title : Word Representation Techniques

Aim

To implement and train a basic word embedding model using a limited text corpus in Python in order to learn vector representations of words and analyze semantic similarities and relationships among them.

Objectives

1. To preprocess and prepare a text corpus for training word embeddings.
2. To implement a basic word embedding technique (such as Word2Vec or CBOW/Skip-gram model) in Python.
3. To measure semantic similarity between words using vector distance metrics like cosine similarity.
4. To analyze and interpret semantic relationships captured by the trained word embeddings.

Expected Outcomes

1. Successful training of basic word embeddings from the given text corpus.
2. Generation of meaningful vector representations for each word in the vocabulary.
3. Ability to compute semantic similarity between words using measures like cosine similarity.
4. Identification of semantic relationships (e.g., similar or related words appearing closer in vector space).
5. Improved understanding of how word embeddings capture contextual meaning in Natural Language Processing (NLP).

Theory

1. Introduction to Word Embeddings

Word Embedding is a technique in Natural Language Processing (NLP) used to represent words as dense numerical vectors in a continuous vector space. Unlike traditional one-hot encoding, where each word is represented by a sparse vector with mostly zeros, word embeddings capture semantic meaning and relationships between words.

For example:
Words like "king" and "queen" or "cat" and "dog" will have similar vector representations because they appear in similar contexts.

Word embeddings are widely used in:

- Sentiment analysis

- Text classification
- Machine translation
- Chatbots
- Recommendation systems

2. Distributional Hypothesis

The foundation of word embeddings is the Distributional Hypothesis, which states:

"Words that appear in similar contexts tend to have similar meanings."

For example, the words *"car"* and *"bike"* may appear in similar contexts such as transportation, travel, and road. Therefore, their vector representations will be closer in the embedding space.

This idea helps machines understand semantic similarity based on contextual usage.

3. Word Representation Techniques

1.Dictionary Lookup

Assign a unique ID (index) to each unique word in the vocabulary.

Representation:

$$\text{word} \rightarrow \text{index}$$

If vocabulary size = V

$$\text{Vocabulary} = \{w_1, w_2, \ldots, w_V\}$$

Mapping:

$$w_i \rightarrow i$$

2.One-Hot Encoding

Each word is represented as a vector of size V(vocabulary size).
Only one position = 1, rest = 0.

Formula:

For word w_i:

$$\mathrm{x}_i = [0,0,0,\ldots,1,\ldots,0]$$

Where:

$$x_j = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

3.Bag of Words (BoW)

Idea:
Represents a document using word frequencies.

If vocabulary size = V

Document vector:

$$\mathrm{d} = [c_1, c_2, \ldots, c_V]$$

Where:

$$c_i = \text{Count of word } w_i \text{ in document}$$

4.TF-IDF (Term Frequency - Inverse Document Frequency)

Gives weight based on importance of word in document and rarity across corpus.

Step 1: Term Frequency (TF)

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in } d}{\text{Total terms in } d}$$

Step 2: Inverse Document Frequency (IDF)

$$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

Where:

- N= Total number of documents
- $df(t)$= Number of documents containing term t

Final TF-IDF Formula

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

4. Word2Vec Model

Word2Vec is a popular technique used to generate word embeddings. It has two main architectures:

- **Continuous Bag of Words (CBOW)**

Predicts the target word from surrounding context words. Continuous Bag of Words (CBOW) is a neural network-based model used to predict a target word given its surrounding context words. This model focuses on the context around a word to predict the word itself, which is useful for learning meaningful word representations. CBOW uses unsupervised learning, meaning it learns from large collections of text without requiring labeled data.

Example:
Context: “I love ___”
Prediction: “python”

- **Skip-gram**

Predicts surrounding context words from a target word. The Skip-gram model is the reverse of CBOW. Instead of predicting the target word based on its context, Skip-gram predicts the context words given the target word. It tries to maximize the probability of predicting the surrounding words for a given target word. This model captures richer word representations, especially for rare words.

Example:
Target word: “python”
Prediction: “I”, “love”, “coding”

During training, the model adjusts word vectors using optimization techniques such as gradient descent to reduce prediction error.

5. Advantages of Word Embeddings

1.Captures semantic meaning
2.Reduces dimensionality.
3.Improves NLP model performance
4.Learns relationships automatically

6. Measuring Semantic Similarity

Semantic similarity between words is measured using **Cosine Similarity**:

Cosine Similarity = $(A \cdot B) / (||A||\ ||B||)$

Where:

- A and B are word vectors
- Value close to 1 → High similarity

- Value close to 0 → Low similarity

Example:
Similarity("king", "queen") → High
Similarity("king", "table") → Low

Pseudocode

```
START
INPUT:
    Text corpus
    Embedding dimension (D)
    Window size (context size)
    Number of epochs

STEP 1: Preprocess Text
    1. Convert text to lowercase
    2. Remove punctuation and special characters
    3. Tokenize text into words
    4. Store words in a list

STEP 2: Build Vocabulary
    1. Identify unique words from corpus
    2. Assign an index to each word
    3. Store vocabulary size (V)

STEP 3: Generate Training Data (Using Skip-gram or CBOW)
    IF Skip-gram:
        FOR each word in corpus
            Take target word
            Select surrounding words within window size
            Create (target, context) pairs
        END FOR

    IF CBOW:
        FOR each word in corpus
            Take surrounding words as input
            Target word as output
        END FOR

STEP 4: Initialize Model
    1. Initialize input weight matrix (V × D) randomly
    2. Initialize output weight matrix (D × V) randomly
```

```
STEP 5: Train Model
    FOR each epoch
        FOR each training pair
            1. Forward pass: Compute predicted output
            2. Calculate loss (error)
            3. Backpropagate error
            4. Update weight matrices using gradient descent
        END FOR
    END FOR

STEP 6: Generate Word Embeddings
    Extract trained input weight matrix
    Each row represents embedding vector of a word

STEP 7: Compute Similarity
    1. Select two word vectors
    2. Compute cosine similarity
    3. Return similarity score
OUTPUT:
    Trained word embedding vectors
    Semantic similarity between words

END
```

Result

```
--- Starting Word Embedding Training ---
Preprocessing text...
Training Word2Vec model (Skip-gram)...

Model trained successfully! Vocabulary size: 52

Vector representation for 'learning' (Dimension 10):
[-0.017 -0.006  0.066  0.077 -0.077 -0.082  0.117  0.131 -0.09  -0.045]

--- Semantic Similarity Analysis ---
Similarity('artificial', 'intelligence')        -> 0.4745
Similarity('machine', 'learning')       -> 0.6022
Similarity('deep', 'learning')  -> -0.5964
Similarity('language', 'processing')    -> 0.6845
Similarity('learning', 'automation')    -> -0.0410

--- END ---
```

Conclusion

The word embedding model successfully learns dense vector representations of words from a given text corpus. By training using CBOW or Skip-gram architecture, the model captures semantic similarities and relationships between words. Word embeddings improve the machine's ability to understand contextual meaning and serve as a fundamental concept for advanced NLP techniques and deep learning models.